

Integration first with Ballerina

A language that prioritises integration can go a long way towards making your next serverless application seem like a breeze

Mithun Mohandas | mithun@digit.in

B

uilding an app has gotten very easy over the years. Numerous languages have arisen with built-in packages to handle most of the communication legwork, giving you time to focus on the core aspects of your application. However, in doing so, you do end up with too many middleware modules just

use, your communication strategy will have to change. Unless of course, you happen to have a magical power supply that automatically fits into any socket in the world and works with any voltage and frequency. This magical device doesn't exist in the same manner that there isn't a programming language which has integration natively built in. That's what Ballerina aims to be.

ENTER - STAGE RIGHT - BALLERINA

Ballerina comes from the folks over at WSO2, an open source company that focuses on API management, Enterprise Integration, ESB and Identity Management. It is written to be a replacement for the different middleware technologies and has a strong focus towards microservice development. It's essentially cloud native so you can write code without worrying about interpreting different communication standards. For example, it can natively play with JSON, one of the most widely used communication standards in the cloud environment.



Modern day coding in a nutshell

Ballerina

to enable a simple task. Given the fact that these modules are written to encompass an array of features, of which you'll be using one or two, there's a lot more code that you have to be wary of. This also translates to more points of vulnerabilities. As an example, think of it like connecting an Indian electric plug (Type D, 230 V, 50 Hz) of your laptop power brick into an American electric socket (Type B, 120 V, 60 Hz).

Along the way, you need to not only get an adapter which allows the Type D plug to mate with the Type B plug but you also need a step-up transformer to ensure the voltage and frequency are in spec. Some of you might be quick to point out that if your laptop's brick uses a switching mode power supply, then the step-up transformer is not needed. You are right and exactly on point. Depending on the type of middleware you

```
json whodaman = {
    firstname: "Clark",
    lastname: "Kent",
    "age": age,
    address: {
        line: "Fortress
of Solitude",
        location:
"North Pole, Arctic",
        nationality:
"Kandor"
    }
};
io:println(user.address.coun-
try);
```

Similarly, you can mess around with XML as well. Here's an example:

```
xml a1 = xml '<name>Clark
Kent</name>';
io:println(a1);
```

Ballerina is also a static typed language, which means you have to declare the data type. As of now, Ballerina supports boolean, int, float, decimal, string and nil as the basic data types. It also supports non-primitive data types such as Arrays, Tuples, Maps, Unions and Tables. The creators aim to enable developers to tackle fallacies of distributed computing within the application logic.

GETTING STARTED WITH BALLERINA

Ballerina is available for download from their official website (<https://ballerina.io/downloads>) for Windows, Linux and MacOS. There are plugins online to use Ballerina with Visual Studio or IntelliJ IDEA. Ballerina service files have the .bal extension. Here's a simple Hello World service.

```
import ballerina/http;
import ballerina/io;
# A service representing a
network-accessible API
# bound to port `9090`.
```